

O que é o RestAssured?

O **Rest Assured** é uma biblioteca Java muito utilizada para **testes automatizados de APIs REST**. Ela foi criada para facilitar a escrita de testes, deixando o código mais legível e próximo da linguagem natural (estilo *BDD – Behavior Driven Development*). É uma biblioteca que permite:

- Fazer requisições HTTP (GET, POST, PUT, DELETE...)
- Validar respostas da API
- Testar contratos (estrutura do JSON/XML)
- Automatizar testes de integração de APIs

Principais características

Sintaxe fluente (BDD)

O grande diferencial é a leitura fácil:

```
given()
    .baseUrl("http://localhost:8080")
.when()
    .get("/contas")
.then()
    .statusCode(200);
```

Fica parecido com:

- **given** (dado)
- **when** (quando)
- **then** (então)

Suporte a validação de respostas

Você pode validar:

- Status HTTP
- Corpo da resposta (JSON/XML)
- Headers

Exemplo:

```
then()
    .statusCode(200)
```

```
.body("titular", equalTo("João"));
```

Integração com frameworks de teste

Funciona muito bem com:

- **JUnit**
- **TestNG**
- **Hamcrest** (para asserts mais expressivos)

Testes de contrato

Permite validar se a API segue um contrato esperado:

```
then()  
    .body(matchesJsonSchemaInClasspath("schema.json"));
```

Muito útil para garantir que mudanças não quebrem o frontend ou outros consumidores da API.

Suporte a autenticação

Você pode testar APIs protegidas:

```
given()  
    .auth().basic("user", "senha")  
.when()  
    .get("/contas")  
.then()  
    .statusCode(200);
```

Exemplo completo (POST)

```
given()  
    .contentType("application/json")  
    .body("{ \"titular\": \"João\", \"saldo\": 1000 }")  
.when()  
    .post("/contas")  
.then()  
    .statusCode(201)  
    .body("titular", equalTo("João"));
```

Quando usar?

O RestAssured é ideal para:

- ✓ Testes de API REST
- ✓ Testes de integração
- ✓ Testes automatizados em pipelines (CI/CD)
- ✓ Validação de contratos

Vantagens

- Código simples e legível
- Fácil integração com projetos Java (ex: Spring Boot)
- Suporte completo a JSON/XML
- Forte comunidade

Desvantagens

- Focado apenas em Java
- Não substitui testes unitários (atua mais em integração/API)